Sparse & Explainable Bayesian Neural Networks for Official Statistics

Unit-Weighted Pseudo-Posteriors, Formal Differential Privacy, and Non-Probability Propensity Estimation in LBBNN

Aliaksandr Hubin & Lars Skaaret-Lund (NMBU & UiO Integreat)

In collaboration with Terrance D. Savitsky, Matthew R. Williams, Vladislav Beresovsky, and Julie Gershunskaya (U.S. Bureau of Labor Statistics)

October 2026

Section 1

PART I: Foundations of LBBNN & Input-Skip Architecture (ISLaB)

Motivation — Why Sparse & Explainable BNNs in Official Statistics?

The Black-Box & Over-Parameterization Dilemma:

- Modern deep neural networks capture complex multi-way interactions, but lack calibrated uncertainty quantification and interpretable feature attributions.
- Standard dense BNNs quantify parameter uncertainty, but retain all noise covariates, inflating variance in downstream survey estimators.

What Official Statistics Requires:

- **Structural Sparsity**: Combine model selection (spike-and-slab edge pruning) and parameter uncertainty in a single variational framework.
- **Explainable Depth**: Automatically adapt between simple linear GLMs (Depth $= 1$) and deep non-linear interaction surfaces (Depth > 1).
- **Survey & Privacy Rigor**: Rigorous integration with informative sampling weights, differential privacy (ϵ, δ), and non-probability samples.

From Dense BNNs to Latent Binary BNNs (Hubin & Storvik, 2024)

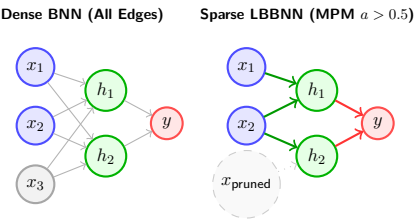
Spike-and-Slab Edge Selection:

- Latent binary inclusion indicator $\gamma_{k,p}^{(j)} \in \{0, 1\}$:

$$p\big(w_{k,p}^{(j)} \mid \gamma_{k,p}^{(j)}\big) = \gamma_{k,p}^{(j)} \mathcal{N}\big(0, (\tau_{k,p}^{(j)})^2\big) + \big(1 - \gamma_{k,p}^{(j)}\big)\delta_0$$

$$p\big(\gamma_{k,p}^{(j)}\big) = \text{Bernoulli}\big(\psi_{k,p}^{(j)}\big)$$

- **Bayesian Model Averaging (BMA)**: Ensembles across posterior draws $\Gamma \sim q(\Gamma)$ and $W \sim q(W \mid \Gamma)$ for optimal uncertainty quantification.
- **Median Probability Model (MPM)**: Hard-prunes edges with posterior inclusion probability $a_{k,p}^{(j)} \leq 0.5$ (Barbieri & Berger, 2004 optimal predictive submodel).



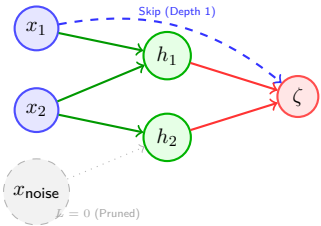
Input-Skip LBBNNs (ISLaB) & Structural Depth Metrics (Høyheim et al., 2026)

The ISLaB Architecture (*JAIR*, 2026):

- Direct skips route inputs $\mathbf{x} \in \mathbb{R}^p$ to every hidden layer: $\mathbf{a}^{(j-1)} = [a_1^{(j-1)}, \ldots, a_K^{(j-1)}, \mathbf{x}^\top]^\top$.
- **Active Path (AP)**: Connects input x_k to output ζ via edges with $a_{k,p}^{(j)} > 0.5$.

Contribution Depth Classification:

- **Depth = 1**: Direct skip $\implies$ linear GLM effect.
- **Depth $\geq$ 2**: Hidden layers $\implies$ non-linear interaction.
- **Paths = 0**: Noise covariate pruned from graph.



Local Explainability via Polyhedral ReLU Activation Regions

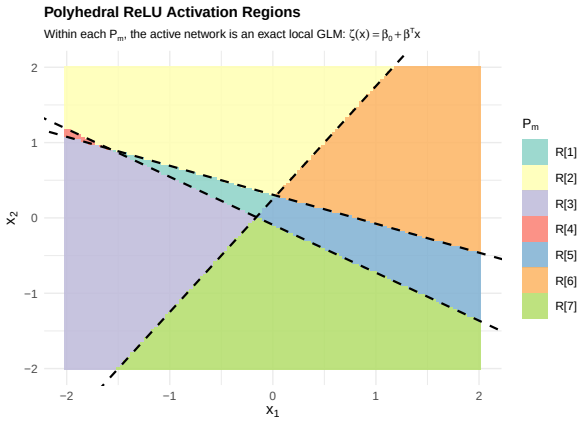
Local GLM Theorem (JSM 2026):

- Piecewise-linear activations partition $\mathbb{R}^p$ into polyhedral activation regions $\mathcal{P}_1, \ldots, \mathcal{P}_M$.
- Within each local region $\mathcal{P}_m$, the active network reduces to an exact **local GLM**:

$$\zeta(\mathbf{x}) = \beta_0(\mathbf{x}_i) + \sum_{j=1}^{p} \beta_j(\mathbf{x}_i) x_j$$

$$\boldsymbol{\beta}(\mathbf{x}_i) = \nabla_{\mathbf{x}} \zeta(\mathbf{x})\big|_{\mathbf{x}=\mathbf{x}_i}$$

- **Exact Gradients**: Evaluated via analytical backpropagation (zero perturbation error).
- **Credible Intervals**: Draws $W \sim q(W \mid \Gamma^*)$ yield exact posterior intervals for respondent sensitivities.



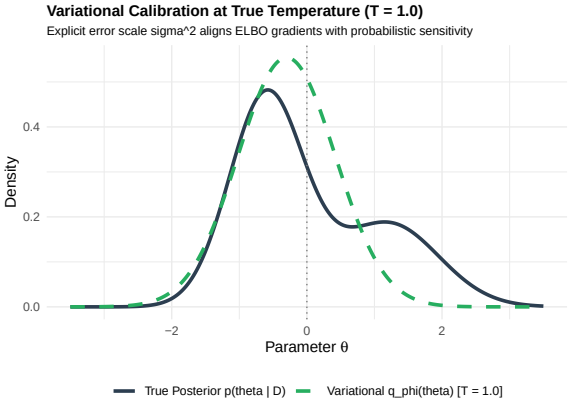
Variational Inference & Protection Against Cold Posteriors

Variational ELBO at True $T = 1.0$:

$$\text{ELBO}(q) = \sum_{i=1}^{n} \mathbb{E}_q[\log f(y_i \mid \mathbf{x}_i, \theta)] - \text{KL}(q \parallel p)$$

Why Explicit Error Scale σ^2 Matters (Wenzel et al., 2020):

- Omitting residual noise scale σ^2 artificially rescales likelihood ($T = \sigma^2 \neq 1$), mis-calibrating spike-and-slab KL shrinkage.
- In `lbbnn_net(sigma = ...)`, calibrated σ^2 grounds log-likelihood sensitivities on the true Bayesian scale.
- Calibrates both feature inclusion probabilities a_k and DP sensitivities f_i.



The LBBNN Open-Source Software Ecosystem

R Package LBBNN (CRAN v0.1.6):

- Pure C++ LibTorch backend via `torch` for R (zero Python dependencies or overhead).
- Mean-field spike-and-slab with Local Reparameterization Trick (LRT) and Normalizing Flows (*TMLR*, 2024).
- Automated model selection, predictive ensembling (BMA), and sparse median probability models (MPM).

Joint BLS Collaboration Modules:

- **Unit Weighting**: `get_dataloaders()` supports non-trainable weights $u_i \geq 0$ with optional sample-size normalization.
- **Residual Calibration**: `lbbnn_net(sigma = ...)` incorporates observation noise scale.
- **Differential Privacy**: `loglik_contributions()` and `fit_dp_lbbnn()` for two-pass (ϵ, δ)-PDP synthesis.
- **Composite Propensities**: Native losses for `STACK`, `LINK1`, `LINK2`, and `MIP`.

Section 2

PART II: Track 1 — Unit-Weighted Pseudo-Posteriors & Formal Differential Privacy

Unit-Weighted Variational Pseudo-Posteriors (Savitsky & Toth, 2016)

Equipping each unit $i \in \{1, \ldots, n\}$ with a non-negative weight $u_i \geq 0$ defines the pseudo-posterior:

$$p_{\mathbf{u}}(\theta \mid \mathbf{y}) \propto p(\theta) \prod_{i=1}^{n} f(y_i \mid \mathbf{x}_i, \theta)^{u_i}$$

$$\text{ELBO}(q; \mathbf{u}) = \sum_{i=1}^{n} u_i \, \mathbb{E}_{q(\theta)}[\log f(y_i \mid \mathbf{x}_i, \theta)] - \text{KL}\big(q(\theta) \parallel p(\theta)\big)$$

1. Survey Sampling Weights ($u_i = \tilde{w}_i$):

- Normalized to sample size n: $\tilde{w}_i = w_i \frac{n}{\sum w_j}$.
- Setting `normalize_weights = TRUE` ensures $\sum \tilde{w}_i = n$ does not overpower spike-and-slab KL shrinkage.
- Corrects informative sampling selection bias.

2. Differential Privacy Weights ($u_i = \alpha_i$):

- Unnormalized weights: $\alpha_i \in [0, 1]$.
- Setting `normalize_weights = FALSE` ensures $\sum \alpha_i \leq n$.
- Downweights high-sensitivity tail outliers and contracts the pseudo-posterior toward the prior.

Track 1A — Informative Survey Sampling Setup & Automated Noise Pruning

Simulation Setup ($N = 2,000, p = 4$):

- **True Non-Linear DGP**:

$$Y_i = 2X_{1i} + 1.5\sin(\pi X_{2i}) + 0.8X_{1i}X_{2i} + \varepsilon_i$$

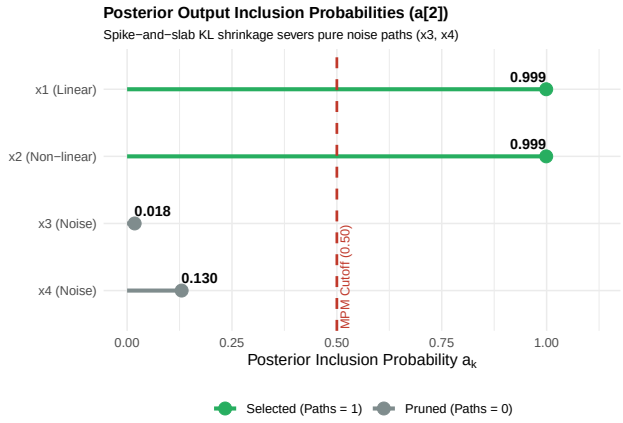
where $\varepsilon_i \sim \mathcal{N}(0, 0.4^2)$, and $X_3, X_4 \sim \mathcal{N}(0, 1)$ are pure noise.

- **Informative Selection Mechanism**:

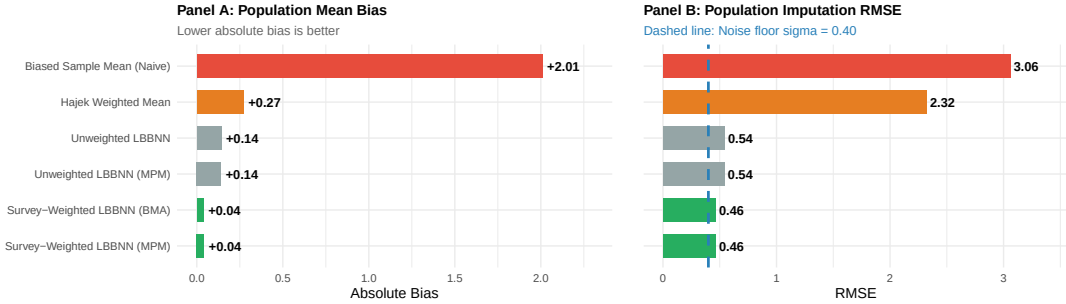
$$\pi_i = \text{logit}^{-1}(-1.0 + 1.2Y_i)$$

Draws $n = 737$ units favoring high Y_i.

- Unweighted sample mean is **+2.009** (True population mean: **0.000**).



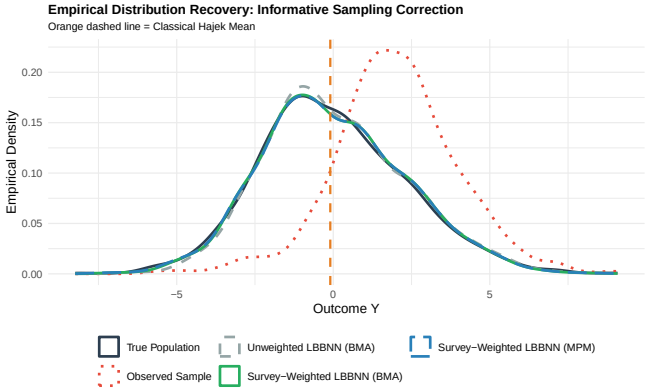
Track 1A — Population Mean & Imputation RMSE Recovery (Visual Benchmark)



Key Takeaways:

- Classical **Hájek mean** reduces bias ($+0.27$) but only provides a single scalar estimate (Pop RMSE $= 2.32$).
- **Survey-Weighted LBBNN** learns the conditional surface, reducing Pop RMSE to **0.46** (approaching $\sigma = 0.40$ noise floor).

Track 1A — Full Population Distribution Recovery



Distributional Recovery: Survey-weighted LBBNN models (green/blue curves) accurately reconstruct the full population density (black solid curve) across all 2,000 units, completely correcting the severe rightward shift of the observed sample (red dotted curve).

Track 1A — Domain Estimation, Imputation & Dirichlet Bootstrap UQ

Domain Totals & De-Noising:

- **Domain Membership Matrix** $\mathbf{G} \in \mathbb{R}^{n \times D_{\text{dom}}}$ defines sub-populations (industries, regions):

$$\hat{\bar{Y}}_d^{\text{BNN}(s)} = \frac{\sum_{i=1}^{n} w_i G_{i,d}\, \hat{\mu}_i^{(s)}}{\sum_{i=1}^{n} w_i G_{i,d}}$$

- **Removing Observation Noise**: Using latent conditional mean $\hat{\mu}_i^{(s)} = \mu_{\theta^{(s)}}(\mathbf{x}_i)$ explicitly strips additive measurement error $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, recovering pure regression surfaces.
- **Item Non-Response Imputation**: Missing outcomes ($y_i = \text{NA}$) are filled via posterior predictive draws $\tilde{y}_i^{(s)} \sim \mathcal{N}(\mu_{\theta^{(s)}}(\mathbf{x}_i), \sigma^2)$ (`impute_response_lbbnn`).

Dirichlet Bayesian Bootstrap UQ:

- **The Pseudo-Posterior Variance Deficit**: Pseudo-posteriors condition on fixed sample S, underestimating design-based sampling variance.
- **Dirichlet Perturbations** (Savitsky & Toth, 2016): For each draw $s \in \{1, \ldots, S\}$, sample flat Dirichlet perturbations:

$$\boldsymbol{\xi}^{(s)} = (\xi_1^{(s)}, \ldots, \xi_n^{(s)}) \sim \text{Dirichlet}(1, \ldots, 1)$$

$$\hat{\bar{Y}}_d^{(s)} = \frac{\sum_{i=1}^{n} \xi_i^{(s)} w_i G_{i,d}\, y_i^{(s)}}{\sum_{i=1}^{n} \xi_i^{(s)} w_i G_{i,d}}$$

- Restores exact finite-population frequentist coverage in credible intervals (`dirichlet_uq = TRUE`).

Pipeline & Return Structures: Survey-Weighted LBBNN (Track 1A)

```r
# 1. Package design weights & allow missing response values (y_i = NA)
dl_train <- get_dataloaders(df_survey, weights = w_design, normalize_weights = TRUE,
                            allow_missing = TRUE, train_batch_size = 64, test_batch_size = 32)
# 2. Build LBBNN with calibrated residual noise scale sigma (protects T=1)
net <- lbbnn_net(problem_type = "regression", sizes = c(p, 16, 1), sigma = 0.4, input_skip = TRUE)
fit <- train_lbbnn(LBBNN = net, train_dl = dl_train$train_loader, epochs = 100, lr = 0.01)
# 3. Domain predictions & Bayesian imputation with Dirichlet uncertainty draws
dom_est <- predict_domains_lbbnn(net, X, domain_matrix = G, weights = w_design,
                                 type = "latent_mean", dirichlet_uq = TRUE, draws = 50)
imp_res <- impute_response_lbbnn(net, newdata = df_survey, target_col = "y", draws = 50)
```

Step / Object	R Class / Type	Dimensions	Statistical Contents & Role
`dl_train$train_loader`	`torch::dataloader`	Batches: $B = 64$	Tensors `x` $[B, p]$, `y` $[B, 1]$, and weights `w` $[B, 1]$ ($\sum \tilde{w}_i = n$).
`net`	`lbbnn_net` (R6)	$p \to 16 \to 1$	Variational weights μ_W, σ_W, logits ψ; direct skips $\mathbf{x} \to h_j, \zeta$.
`dom_est`	`data.frame`	$D_{\text{dom}} \times 10$	Domain means, totals, posterior SDs, and 95% Dirichlet credible bo
`imp_res$completed_data`	`data.frame`	$n \times (p + 1)$	Completed dataset with missing y_i replaced by posterior mean $\hat{\mu}_i$

- **Survey Normalization**: `normalize_weights = TRUE` enforces $\sum \tilde{w}_i = n$ (Savitsky & Toth, 2016).
- **Latent Mean De-Noising**: `type = "latent_mean"` eliminates observation error $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$.

Track 1B — Why Unweighted Neural Networks Fail Differential Privacy

Probabilistic Differential Privacy (Savitsky, Williams & Hu, 2022):

$$\sup_{\theta \in \Theta_\delta} \left| \log \frac{q_{\boldsymbol{\alpha}}(\theta \mid \mathbf{y})}{q_{\boldsymbol{\alpha}}(\theta \mid \mathbf{y}')} \right| \leq \epsilon_{\mathbf{y}} = 2\Delta_{\alpha,\mathbf{y}}, \qquad \Delta_{\alpha,\mathbf{y}} = \max_{s,i} |\alpha_i \log f(y_i \mid \mathbf{x}_i, \theta^{(s)})|$$

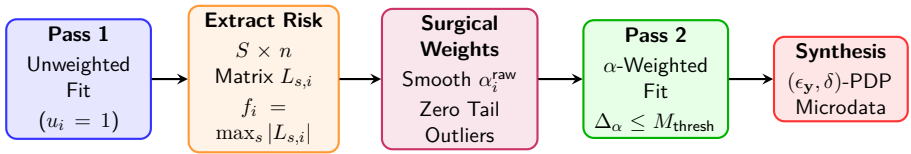
The Quadratic Exposure Dilemma in Gaussian Regression:

- Pointwise Gaussian log-likelihood grows quadratically with the residual:

$$|\log f(y_i \mid \mathbf{x}_i, \theta)| \propto \frac{(y_i - \mu_\theta(\mathbf{x}_i))^2}{2\sigma^2}$$

- In skewed survey microdata (wages, financial expenditures), extreme outliers drive unweighted sensitivity $\Delta \to \infty$ and privacy budget $\epsilon_{\mathbf{y}} \to \infty$.
- Naive gradient clipping distorts the mean and destroys the variance structure of official statistics.

Track 1B — The Two-Pass Surgical Downweighting Pipeline (`fit_dp_lbbnn`)

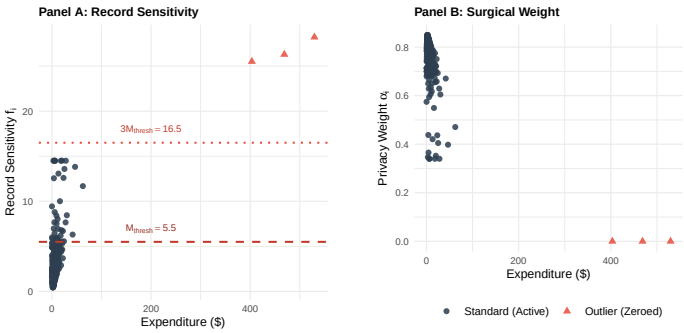


Monotone Surgical Truncation Formula:

$$\alpha_i = \min\left(\alpha_i^{\text{raw}}, \frac{M_{\text{thresh}}}{f_i}\right), \qquad \alpha_i = 0 \quad \text{if } f_i > 3M_{\text{thresh}}$$

Guarantees local Lipschitz bound $\Delta_{\alpha,\mathbf{y}} = \max_{s,i} |\alpha_i L_{s,i}^{(2)}| \leq M_{\text{thresh}}$ and bounded privacy $\epsilon_{\mathbf{y}} = 2\Delta_{\alpha,\mathbf{y}}$.

Track 1B — Monotone Surgical Downweighting Diagnostic



Empirical DP Metrics:

- **Sample**: $n = 300$, median $\$4.16$, 3 extreme outliers ($\$400$–$\$530$).
- **Standard Records**: Retain high weights $\alpha_i \in [0.34, 0.85]$ (median 0.81).
- **Tail Outliers**: All 3 outliers ($f_i > 26 > 3M$) surgically zeroed ($\alpha_i = 0.0$).
- **Privacy Guarantee**: $\Delta_{\alpha,\mathbf{y}} = 4.32 \leq 5.50 \implies \boldsymbol{\epsilon_{\mathbf{y}} = 8.63}$.

Track 1B — Two-Stage Privacy Weight Normalization (Savitsky, 2026)

Record Sensitivity Proxy:

- Extract worst-case empirical log-likelihood sensitivity from unweighted Pass 1:

$$f_i = \max_{s \in \{1,\ldots,S\}} |\ell_i(\theta^{(s)})|, \quad \tilde{\alpha}_i = \frac{1}{f_i}$$

Stage 1: Initial Normalization Options:

1. **Global Linear Min-Max** (`"linear"`):

$$\alpha_i^{\text{lin}} = \text{clip}_{[0,1]}\left(c \cdot \frac{\tilde{\alpha}_i - \tilde{\alpha}_{\min}}{\tilde{\alpha}_{\max} - \tilde{\alpha}_{\min}} + g \right)$$

2. **Percentile Deficit Truncation** (`"percentile"`):

$$\alpha_i^{\text{pct}} = \min\left(1, \max\left(0, \frac{\tilde{\alpha}_i - q_{p_L}(\tilde{\alpha})}{q_{p_R}(\tilde{\alpha}) - q_{p_L}(\tilde{\alpha})}\right)\right)$$

3. **Target Ratio Clipping** (`"target_clip"`):

Theoretical Properties (Proposition 1):

- **Min-Max Vulnerability**: Inliers ($f_{j^*} \to 0 \implies \tilde{\alpha}_{\max} \to \infty$) uniformly deflate intermediate sample weights.
- **Percentile Deficit**: Truncates bottom p_L tail risk and boosts intermediate sample size ($\sum \alpha_i$), but requires Stage 2 to bound $\alpha_i f_i$.
- **Exact Sensitivity Control**: Target clipping guarantees $\alpha_i^{\text{clip}} f_i = \min(f_i, \Delta_{\text{goal}}) \leq \Delta_{\text{goal}}$ by construction, leaving all low-sensitivity records untouched at $\alpha_i = 1$.

Track 1B — Stage 2: Hard Truncation vs. Vector-Weighted Upward Rescaling

Scheme 1: Hard Truncation (`"hard_trunc"`):

- Post-hoc zeroing of records exceeding threshold:

$$\alpha_i^{\text{trunc}} = \alpha_i \, \mathbb{I}(\alpha_i f_i \leq M_{\text{thresh}})$$

- **Limitation**: Single-pass truncation completely discards tail observations and leaves unspent privacy slack $M_{\text{thresh}} - \alpha_i f_i > 0$.

Scheme 2: Vector Rescaling (`"vector_upscale"`):

- Upward rescaling to full threshold capacity:

$$\alpha_i^{\text{vec}} = \min\left(1, \, \alpha_i \cdot \frac{M_{\text{thresh}}}{\alpha_i f_i}\right) = \min\left(1, \, \frac{M_{\text{thresh}}}{f_i}\right)$$

Information Dominance (Proposition 2):

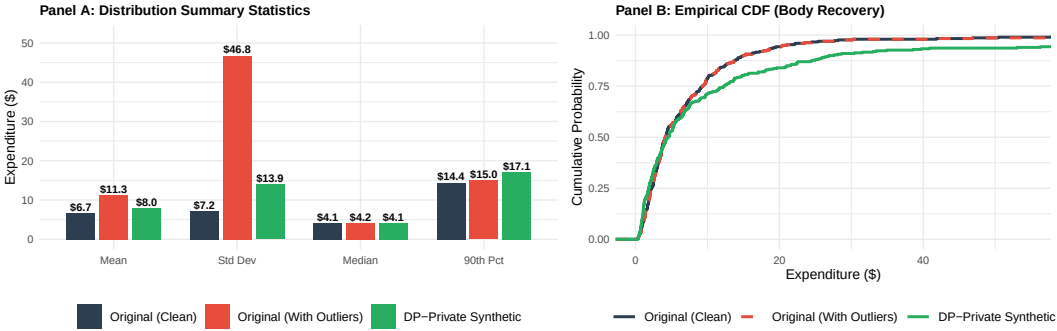
- Because $\alpha_i^{\text{vec}} \geq \alpha_i^{\text{trunc}}$ everywhere, vector upward rescaling strictly maximizes Fisher information:

$$\mathcal{I}_{\boldsymbol{\alpha}^{\text{vec}}}(\theta) \succeq \mathcal{I}_{\boldsymbol{\alpha}^{\text{trunc}}}(\theta)$$

in the Loewner partial order.

- **Pass 3 Execution**: Upward rescaling shifts pseudo-posterior geometry $\xi_{\alpha^{\text{vec}}}(\theta \mid D)$; `fit_dp_lbbnn` executes an additional variational pass targeting $\xi_{\alpha^{\text{vec}}}$ to ensure calibration and bound verification.

Track 1B — Synthetic Microdata Utility vs. Outlier Protection



Distributional Utility vs. Disclosure Protection:

- 3 extreme outliers distort raw sample mean ($\$11.30$ vs. $\$6.70$) and explode standard deviation to $\$46.75$.
- **DP-Private Synthetic** microdata recovers the true distribution body (Mean $\$7.99$, Median $\$4.13$) while safely truncating disclosure risk on extreme tail outliers.

Pipeline & Return Structures: Two-Pass DP Synthesis (Track 1B)

```r
# 1. Calibrate privacy weights: Stage 1 target clipping + Stage 2 vector upward rescaling
dp_w <- compute_dp_weights(f_sens, stage1_method = "target_clip", stage2_method = "vector_upscale",
                           M_thresh = 5.5, Delta_goal = 5.5)
# 2. End-to-end multi-pass DP estimation (executes Pass 3 for vector upscale geometry)
dp_fit <- fit_dp_lbbnn(model_spec = list(sizes = c(p, 16, 1), problem_type = "regression"),
                       X = x_micro, y = y_sensitive, stage1_method = "target_clip",
                       stage2_method = "vector_upscale", M_thresh = 5.5, epochs = 100, lr = 0.01)
# 3. Generate private synthetic microdata via post-processing theorem
synth_micro <- sample_synthetic_lbbnn(dp_fit, newdata = x_micro, draws = 50)
```

Step / Object	R Class / Type	Dimensions	Statistical Contents & Role
`dp_fit$model`	`lbbnn_net` (R6)	$p \to 16 \to 1$	Final private model targeting calibrated pseudo-posterior $\xi_{\boldsymbol{\alpha}^{\text{vec}}}$.
`dp_fit$alpha`	`numeric vector`	$[n]$ records	Upward-rescaled weights $\alpha_i^{\text{vec}} = \min(1, M_{\text{thresh}}/f_i) \in [0, 1]$.
`dp_fit$epsilon`	`numeric scalar`	Scalar $\epsilon_{\mathbf{y}}$	Finite-sample local DP bound $\epsilon_{\mathbf{y}} = 2\Delta_{\alpha,\mathbf{y}} \leq 2M_{\text{thresh}}$.
`synth_micro`	`matrix`	$[n, 50]$	50 private synthetic replicates drawn with calibrated $\hat{\sigma}$.

- **Loewner Dominance**: `"vector_upscale"` pointwise dominates hard truncation and maximizes Fisher information.
- **Post-Processing Theorem**: Downstream analysts query `synth_micro` with zero disclosure risk.

Section 3

PART III: Track 2 — Combining Probability & Non-Probability Samples

The Big Data Paradox in Convenience Samples (Meng, 2018)

The Non-Probability Selection Dilemma:

- Let $U = \{1, \ldots, N\}$ denote the finite target population of size N.
- In convenience/opt-in samples $S_c \subset U$, sample inclusion $I_{ci} \in \{0, 1\}$ is non-random and driven by unobserved self-selection mechanisms.
- **Meng's Law of Large Populations (2018)**:

$$\text{Bias}(\bar{y}_c) = \frac{\text{Cov}_U(I_c, Y)}{\bar{\pi}_c} = \rho_{I_c,Y} \cdot \sigma_Y \cdot \sqrt{\frac{N - n_c}{n_c}}$$

- **Exact Definitions**:
  - $\pi_{ci} = \mathbb{P}(I_{ci} = 1 \mid \mathbf{x}_i)$: Unknown propensity that unit i with covariates $\mathbf{x}_i$ self-selects into S_c.
  - $\bar{\pi}_c = \frac{1}{N} \sum_{i \in U} \pi_{ci}$: Average participation propensity across the entire target population U.
- A tiny correlation $\rho_{I_c,Y} \neq 0$ creates selection bias scaling with $\sqrt{N}$; increasing sample size n_c merely concentrates error around a severely biased estimate.

The Identification Barrier & The Conditioning Trick (Beresovsky et al., 2026)

The Identification Barrier:

- Covariates $\mathbf{x}_i$ are observed only on convenience sample S_c (unknown π_{ci}) and reference survey S_r (known design probability π_{ri}). Non-sampled units $U \setminus (S_c \cup S_r)$ are never observed.

The Conditioning Trick & Structural Offset Derivation:

- Pool S_c and S_r into a combined sample S, and model membership indicator $D_i = \mathbb{I}(i \in S_c)$:

$$\pi_{zi}^{\mathsf{STACK}} = \mathbb{P}(D_i = 1 \mid \mathbf{x}_i, i \in S) = \frac{\pi_{ci}}{\pi_{ci} + \pi_{ri}}$$

- **Step-by-Step Log-Odds (Logit) Expansion**:

$$\log\left(\frac{\pi_{zi}^{\mathsf{STACK}}}{1 - \pi_{zi}^{\mathsf{STACK}}}\right) = \log\left(\frac{\pi_{ci}}{\pi_{ri}}\right) = \log \pi_{ci} - \log \pi_{ri} = \eta_\theta(\mathbf{x}_i) - \log \pi_{ri}$$

- **Exact Structural Offset**: Because π_{ri} is known by survey design, $-\log \pi_{ri}$ enters as an **exact, fixed offset** in the binary cross-entropy loss, anchoring unconstrained neural

Intuition & Torch Training Link for All Five Estimators

Estimator	Sample Overlap	Composite Prob π_{zi}	Torch Backprop & Loss Link
`STACK`	$S_c \oplus S_r$ (Unlinked)	$\frac{\pi_{ci}}{\pi_{ci}+\pi_{ri}}$	Offset logit$(\pi_{zi}) = \eta_\theta(\mathbf{x}_i) - \log \pi_{ri}$ in Bernoulli NLL.
`LINK1`	$S_c \cup (S_r \setminus S_c)$	$\frac{\pi_{ci}}{\pi_{ci}+\pi_{ri}-\pi_{ci}\pi_{ri}}$	Deduplicated union; backprops through $-\pi_{ci}\pi_{ri}$ overlap adjustment.
`LINK2`	$S_c \oplus S_r$ (Linked Stack)	$\frac{\pi_{ci}(1+\pi_{ri})}{\pi_{ci}+\pi_{ri}}$	Exact indicator $I_{zi} = D_i + (1-D_i)I_{ci}$; maximizes Fisher info.
`MIP`	Missing Information	Bridges `STACK` & `LINK2`	Evaluates expectation $\tilde{I}_{zi}^{\text{MIP}} = D_i + (1-D_i)\tilde{p}_i$ with $\tilde{p}_i \in [0,1]$.
`PSEUDO`	Survey-Weighted	Design pseudo-likelihood	Score: $\sum_{S_r} \pi_{ri}^{-1}$softplus$(\eta_i) + \sum_{S_c} \log(1+e^{-\eta_i}) = 0$.

- **Composite vs. Pseudo**: Composite likelihoods (`STACK`, `LINK1`, `LINK2`, `MIP`) form relative odds fractions that naturally bound loss gradients, whereas `PSEUDO` multiplies unconstrained logits by π_{ri}^{-1}.

Where Marginal Linkage Weights $\tilde{p}_i$ Come From in MIP

Probabilistic Record Linkage Origin:

- In agency applications, deterministic matching is often precluded by privacy regulations or noisy identifiers (names, addresses).
- An auxiliary probabilistic record linkage model (e.g., Fellegi-Sunter) estimates pairwise match probabilities:

$$m_{ij} = \mathbb{P}(i \sim j \mid \mathbf{x}_i, \mathbf{x}_j) \in [0, 1]$$

- Marginalizing across reference candidates yields each unit's linkage probability:

$$\tilde{p}_i = \sum_{j \in S_r} m_{ij} \in [0, 1]$$

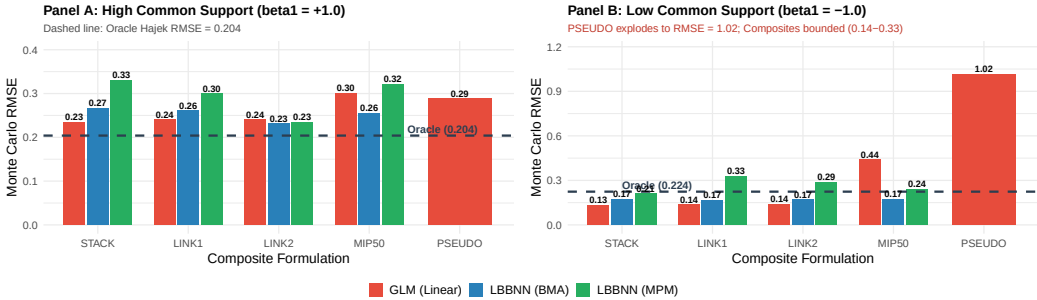
Vectorized Unit Linkage in 'LBBNN':

- Analysts supply the unit-level linkage vector:

$$\tilde{\mathbf{p}} = (\tilde{p}_1, \ldots, \tilde{p}_n)^\top$$

  directly into `make_nll_mip(p_link = p_vec)`.
- **Adaptive Gradient Pull**:
  - High-certainty matches ($\tilde{p}_i \approx 1$) pull gradient updates toward the low-variance `LINK2` surface.
  - Ambiguous/unmatched units ($\tilde{p}_i \approx 0$) safely revert to the robust `STACK` formulation.
- Smoothly bridges deterministic linkage and unlinked stacking record-by-record.

Study 0 — Exact Linear Baseline Monte Carlo ($R = 20$, Visual Benchmark)



Empirical Findings, The Oracle Benchmark, & Stein's Phenomenon:

- **The Oracle Benchmark**: Theoretical bound plugging true data-generating propensities π_{ci} directly into Hájek weighting (bypassing propensity estimation).
- **Why LBBNN Beats the Oracle in Panel B (Low Support)**: Under low common support, true propensities near 0 create extreme $1/\pi_{ci}$ weights and high sampling variance. `LBBNN` regularization smooths tail weights, trading tiny bias for a large variance reduction (Stein's phenomenon).
- **Pseudo-Likelihood Breakdown**: Under low support, `GLM-PSEUDO` explodes to

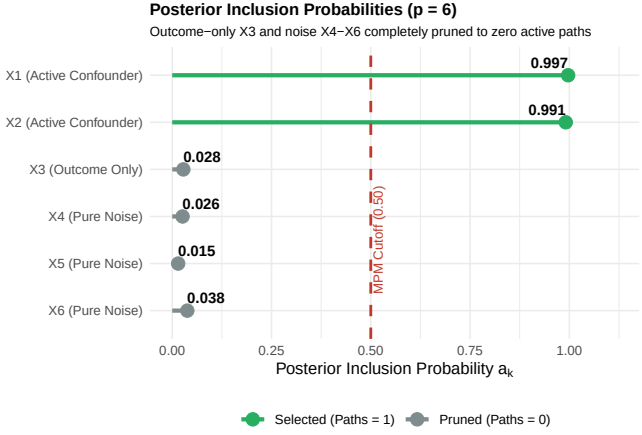
Study 1 — Automated Confounder Selection & Noise Pruning ($p = 6$)

Simulation Setup ($N = 5,000, p = 6$):

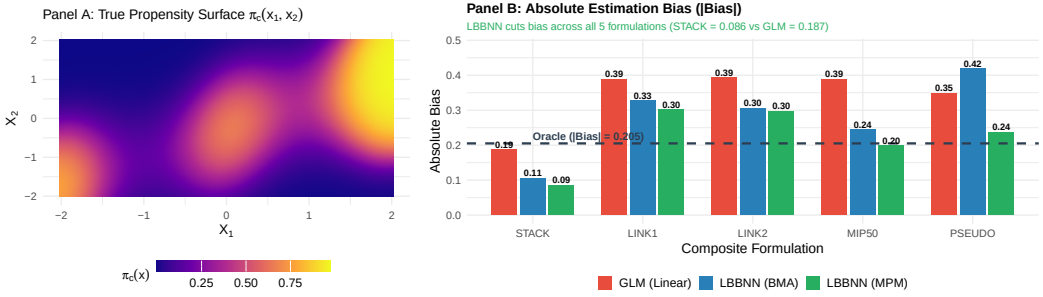
- **Confounders**: X_1, X_2 drive both propensity π_{ci} and outcome Y_i.
- **Outcome-Only**: X_3 drives Y_i but uncorrelated with selection I_c.
- **Pure Noise**: $X_4, X_5, X_6 \sim \mathcal{N}(0, 1)$.

Why Variable Selection Matters:

- In propensity modeling, forcing outcome-only X_3 does **not** reduce selection bias, but **inflates Hájek weight variance**.
- `LBBNN` automatically prunes X_3 ($\alpha_3 = 0.028 \ll 0.50$) along with pure noise X_4–X_6, isolating true



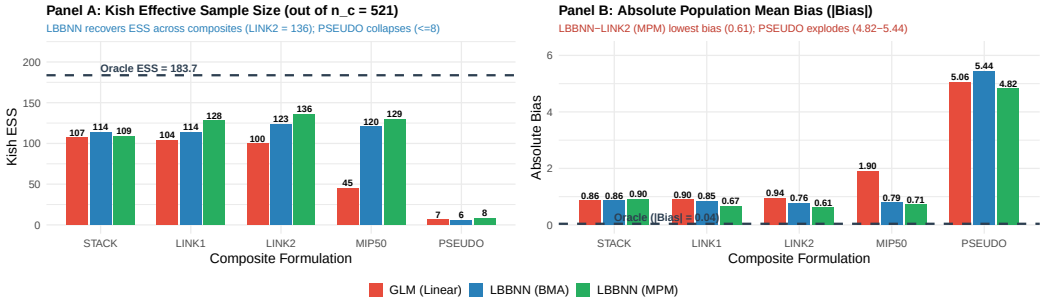
Study 2 — High Common Support with Non-Linear Propensity & Interaction



Non-Linear Recovery Across Formulations:

- Linear GLMs are structurally misspecified on the interaction surface ($|\text{Bias}| = 0.19$–0.39 across formulations).
- `LBBNN` (BMA and MPM) captures non-linear interactions, reducing bias across all 5 formulations (lowest on `STACK`: $|\text{Bias}| = 0.086$ vs GLM 0.187; `MIP50`: 0.198 vs 0.229; `PSEUDO`: 0.237 vs 0.347).

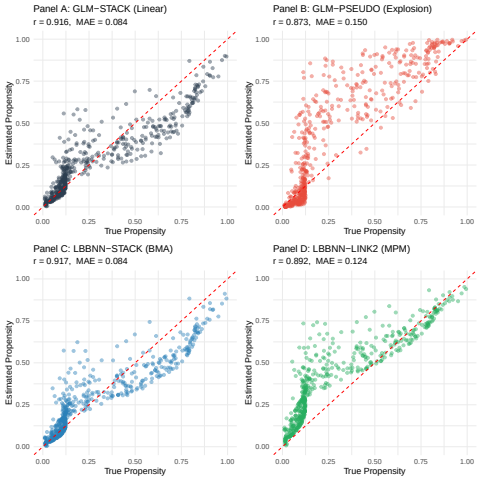
Study 3 — Low Common Support + Non-Linearity: Bias vs. Kish's ESS



Catastrophic Breakdown of Pseudo-Likelihood vs. Composite LBBNN:

- `PSEUDO` collapses Kish ESS (ESS $= 6.6$ out of $n_c = 521$) and explodes bias ($|\text{Bias}| = 4.82$–5.44) due to inverse weight variance and gradient explosion.
- Composite formulations remain stable; `LBBNN-LINK2 (MPM)` achieves the **lowest bias (+0.61)** and **highest effective sample size (Kish ESS = 136.0)**.
- On `MIP50`, LBBNN restores ESS from 45.1 to 129.4 and cuts bias from 1.90 to 0.71.

Study 3 — Propensity Recovery Diagnostic (2x2 Scatterplot Grid)



Panels: (A) GLM-STACK bias from missing interactions; (B) GLM-PSEUDO severe weight explosion and scatter collapse; (C) LBBNN-STACK Bayesian Model Averaging calibration; (D) LBBNN-LINK2 Median Probability Model point recovery.

Pipeline & Return Structures: Composite Propensity (Track 2)

```r
# 1. Format combined sample with vectorized linkage probabilities p_vec
prep    <- build_propensity_dataset(pop, method = "MIP", covar_cols = paste0("X", 1:p), p_link = p_vec_r)
dl_prop <- get_dataloaders(prep$data, target_cols = prep$target_cols, train_batch_size = 64, test_batch_s
# 2. Fit LBBNN with composite Bernoulli loss anchored by pi_ri offset
net_prop <- lbbnn_net(problem_type = "custom", sizes = c(p, 8, 1), raw_output = TRUE, nll = make_nll_mip(
train_lbbnn(LBBNN = net_prop, train_dl = dl_prop$train_loader, epochs = 150, lr = 0.02)
# 3. Predict convenience propensities pi_c_hat & compute debiased Hajek mean
logits_c    <- predict(net_prop, newdata = prep$data[prep$Sc_indices, 1:p], mpm = TRUE)
pi_c_hat    <- as.numeric(torch::torch_sigmoid(torch::torch_tensor(logits_c)))
y_bar_hajek <- hajek_estimate(y = pop$y[pop$I.c == 1], pi_hat = pi_c_hat, dirichlet_uq = TRUE)
```

Step / Object	R Class / Type	Dimensions	Statistical Contents & Role
prep$data	data.frame	$[n_c + n_r, p + 4]$	Covariates $\mathbf{X}$ and composite target columns $[D_i, I_{ci}, \pi_{ri}, \tilde{p}_i]$.
net_prop	lbbnn_net (R6)	$p \to 8 \to 1$	Custom net with composite Bernoulli loss and structural offset π_{ri}.
pi_c_hat	numeric vector	$[n_c]$	Estimated convenience propensities $\hat{\pi}_{ci} \in (0, 1)$ on S_c.
y_bar_hajek	data.frame	Summary table	Debiased Hájek mean with 95% Dirichlet Bayesian bootstrap credib interval.

- **Vectorized Linkage**: `make_nll_mip()` ingests unit-level linkage vector $\tilde{\mathbf{p}}$ elementwise during batching.
- **Loss Stability**: Relative odds fractions naturally bound gradients, avoiding `PSEUDO` explosion.

Discussion — Bridging Track 2 Propensity Weights with Track 1 Regression

Two-Stage Quasi-Randomization (Track 2):

- **Stage 1**: `LBBNN` propensity model estimates $\hat{\pi}_{ci} = \mathbb{P}(I_{ci} = 1 \mid \mathbf{x}_i)$.
- **Stage 2**: Apply outcome-agnostic Hájek weights:

$$\hat{w}_i = \hat{\pi}_{ci}^{-1}, \qquad \hat{\bar{Y}}_{\text{Hajek}} = \frac{\sum_{i \in S_c} \hat{w}_i y_i}{\sum_{i \in S_c} \hat{w}_i}$$

  to any survey response variable y_i.
- **Outcome-Only Sparsity**: Pruning X_3 (which predicts Y but not I_c) prevents weight variance inflation.

Doubly Robust Pseudo-Posterior Imputation:

- **Bridging Track 2 $\rightarrow$ Track 1**: Normalize inverse propensity weights to sample size:

$$\tilde{w}_i = \hat{\pi}_{ci}^{-1} \frac{n_c}{\sum_{j \in S_c} \hat{\pi}_{cj}^{-1}}$$

  and feed $u_i = \tilde{w}_i$ directly into a **Track 1 LBBNN regression model** for y_i.
- **Double Robustness**: Consistent if *either* propensity model *or* outcome regression is correctly specified.
- Outcome model retains X_3 via spike-and-slab, recovering optimal predictive accuracy!

Section 4

PART IV: Software Architecture, Key Takeaways & Synthesis

Unified LBBNN Architecture Across Track 1 & Track 2

Function / Module	Operational Track	Description & Statistical Role
`get_dataloaders()`	Track 1 & 2	Handles non-trainable weights $u_i \geq 0$, normalization, missing y_i, and multi-targets.
`lbbnn_net(sigma, ...)`	Track 1 & 2	ELBO likelihood scaling by error variance σ^2 (cold-posterior protection).
`predict_domains_lbbnn()`	Track 1A (Surveys)	Vectorized domain means/totals, latent mean de-noising ($\hat{\mu}_i$), and Dirichlet UQ.
`impute_response_lbbnn()`	Track 1A (Surveys)	Bayesian multiple imputation for missing outcomes (y_i = NA) via posterior predictive draws.
`compute_dp_weights()`	Track 1B (Privacy)	Modular Stage 1 (`linear`, `percentile`, `target_clip`) × Stage 2 (`hard_trunc`, `vector_upscale`).
`fit_dp_lbbnn()`	Track 1B (Privacy)	Multi-pass (ϵ, δ)-PDP estimation; executes Pass 3 under vector upward rescaling.
`sample_synthetic_lbbnn()`	Track 1B (Privacy)	Generates (ϵ, δ)-DP synthetic microdata via the post-processing theorem.
`make_nll_mip(p_link)`	Track 2 (Propensity)	Missing Information Principle NLL supporting scalar or vectorized unit-level linkage $\tilde{\mathbf{p}}$.
`nll_link1()` / `nll_link2()`	Track 2 (Propensity)	Composite Bernoulli NLL on singly matched and matched-pair samples.
`nll_stack()` / `nll_pseudo()`	Track 2 (Propensity)	Stacked pooled sample NLL and survey design-weighted pseudo-NLL.
`build_propensity_dataset()`	Track 2 (Propensity)	Formats covariates and targets for any of the 5 estimation methods.
`hajek_estimate()`	Track 1 & 2	Self-normalizing Hájek mean with optional domain matrix $\mathbf{G}$ and Dirichlet bootstrap UQ.

Key Takeaways & Synthesis for Official Statistics

1. **Survey-Weighted Pseudo-Posteriors (Track 1A)**:
   - Normalizing survey weights to sample size ($\sum \tilde{w}_i = n$) + calibrated observation noise σ^2 prevents likelihood mis-scaling ($T \neq 1$) and protects spike-and-slab KL shrinkage.
   - `LBBNN` achieves 5x lower population RMSE than classical Hájek weighting under informative sampling while pruning irrelevant covariates.
2. **Two-Pass (ϵ, δ)-PDP Microdata Synthesis (Track 1B)**:
   - Monotone surgical downweighting and vector upward rescaling (`fit_dp_lbbnn`) eliminate disclosure risk from extreme tail outliers ($\alpha_i = 0$) while preserving high utility ($\alpha_i \in [0.34, 0.85]$) on normal records.
3. **Composite Non-Probability Sample Integration (Track 2)**:
   - Conditioning on pooled sample inclusion $S_c \cup S_r$ allows known design probabilities π_{ri} to anchor neural network logits to the true population scale.
   - Composite Bernoulli likelihoods (`LINK2`, `LINK1`, `MIP`, `STACK`) bound loss gradients and prevent the catastrophic effective sample size collapse (`Kish ESS` < 7) of pseudo-likelihood under low common support.

References & Collaborative Works

1. **Beresovsky, V., Gershunskaya, J., & Savitsky, T. D.** (2024). *Review of quasi-randomization approaches for estimation from non-probability samples*. Statistical Science (arXiv:2312.05383).
2. **Savitsky, T. D., Williams, M. R., & Hu, J.** (2022). *Bayesian pseudo posterior mechanism under asymptotic differential privacy*. Journal of Machine Learning Research, 23(232), 1–37.
3. **Savitsky, T. D., & Toth, D.** (2016). *Bayesian estimation under informative sampling*. Electronic Journal of Statistics, 10(1), 1677–1708.
4. **Savitsky, T. D.** (2026). *Privacy weight normalization and second-stage reweighting strategies in asymptotic differential privacy*. U.S. Bureau of Labor Statistics Report.
5. **Meng, X.-L.** (2018). *Statistical paradises and paradoxes in big data (I): Law of large populations, big data paradox, and the 2016 US presidential election*. The Annals of Applied Statistics, 12(2), 685–726.
6. **Chen, Y., Li, P., & Wu, C.** (2020). *Doubly robust inference with nonprobability survey samples*. Journal of the American Statistical Association, 115(532), 2011–2021.
7. **Skaaret-Lund, L., Høyheim, E., & Hubin, A.** (2026). *LBBNN: An R Package for Sparse and Explainable Bayesian Deep Learning with Latent Binary Bayesian Neural Networks*. The R Journal, 18(3), RJ-2026-048.
8. **Hubin, A., & Storvik, G.** (2024). *Sparse Bayesian neural networks: Bridging model and parameter uncertainty through scalable variational learning*. Mathematics, 12(6), 788.
9. **Skaaret-Lund, L., Storvik, G., & Hubin, A.** (2024). *Sparsifying Bayesian neural networks with latent binary variables and normalizing flows*. Transactions on Machine Learning Research, ISSN: 2835-8856.
10. **Høyheim, E., Skaaret-Lund, L., Sæbø, S., & Hubin, A.** (2026). *Explainable Bayesian deep learning through input-skip Latent Binary Bayesian Neural Networks*. Journal of Artificial Intelligence Research, 86 (arXiv:2503.10496).